

Lab 4: Lowering to an Executable Intermediate Representation (IR)

1. Overview

In this lab, you will implement the lowering phase of a compiler. You will translate a high-level Abstract Syntax Tree (AST) into a low-level three-address Intermediate Representation (IR). The IR will then be executed using a provided interpreter.

2. Learning Objectives

- 1 Understand AST to IR transformation
- 2 Represent control flow explicitly
- 3 Use temporaries to linearize expressions
- 4 Preserve program semantics

3. Compilation Pipeline

```
Source Program
  ↓
AST (from previous labs)
  ↓
[YOU IMPLEMENT]
Lowering (AST → IR)
  ↓
IR Interpreter (provided)
  ↓
Program Output
```

4. What You Need to Implement

Implement the following methods in lowering.py:

```
def lower_program(self, program)
def lower_function(self, fn)
def lower_stmt(self, stmt)
def lower_expr(self, expr)
```

5. Key Rules

- 1 Every expression must return a temporary
- 2 Statements emit instructions (no return value)
- 3 Use fresh temporaries and labels
- 4 All control flow must be explicit

6. IR Instruction Set

Instruction	Meaning
Const(t, c)	t = constant c
Move(dst, src)	assignment
BinOp(dst, op, a, b)	binary operation
Label(L)	label

Jump(L)	unconditional jump
CJump(cond, L1, L2)	conditional branch
Call(dst, f, args)	function call
Return(t)	return value
Print(t)	print value

7. Lowering Examples

Binary Expression

```
# Source: x + y
t1 = add x, y
```

If/Else

```
t1 = lt x, y
cjump t1, L1, L2

L1:
print x
jump L3

L2:
print y
jump L3

L3:
```

While Loop

```
L1:
t1 = lt i, 10
cjump t1, L2, L3

L2:
print i
jump L1

L3:
```

8. Running Your Code

```
python main.py --dump-ir
python main.py --run-ir
python main.py --trace-ir
```

9. Grading/Testing

Run the provided grading tests:

```
pytest test_lab4.py
```

There are 10 tests, each worth 10 points.

10. Debugging Tips

- 1 Use --dump-ir to inspect IR
- 2 Use --trace-ir for step-by-step execution
- 3 Check label correctness
- 4 Verify temporaries are unique

- 5 Ensure control flow is correct

11. Common Mistakes

- 1 Reusing temporaries
- 2 Missing jumps after branches
- 3 Incorrect loop structure
- 4 Returning expressions directly instead of temporaries

12. Submission

The file name must be (your ucm email id).zip/.tar/.rar/.gz. Include a ReadMe describing your implementation and the changes you made.

13. Key Insight

This lab demonstrates how structured programs are transformed into a flat representation where control flow and evaluation order are explicit.

